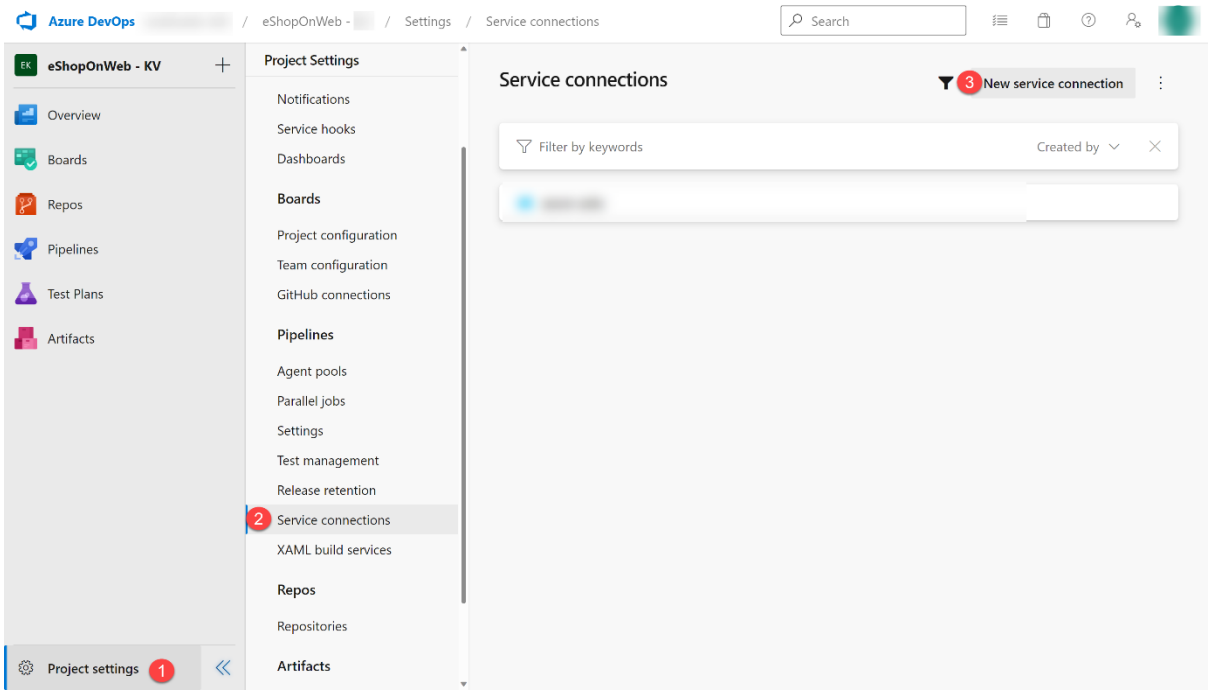
Create a service connection to access Azure resources

You will need to create a service connection in Azure DevOps which will allow you to deploy and access resources in your Azure subscription.

1. Start a web browser, navigate to the Azure DevOps portal https://*aex.dev.azure.com*.
2. Sign in to the Azure DevOps organization with the **Username** and **TAP** present on the **Resources** tab.

Note: If this is the first time you are signing in to the Azure DevOps organization, you will be prompted create your profile and accept the terms of service, and then select **Continue**.

3. Open **eShopOnWeb** project, and select **Project settings** in the bottom left corner of the portal.
4. Select the **Service connections** under Pipelines, and then select **Create service connection** button.



5. On the **New service connection** blade, select **Azure Resource Manager** and **Next** (you may need to scroll down).
6. Select **App registration (automatic)** from the **Identity type** dropbox.
7. Select **Workload Identity federation** and **Subscription** under the **Scope level**.

Note: You can also use **App registration or managed identity (manual)** if you prefer to manually configure the service connection. Follow the steps in the Azure DevOps documentation to create the service connection manually.

8. Fill in the empty fields using the information:
    - **Subscription**: Select your Azure subscription.
    - **Resource group**: Select the resource group **az400m03l08-RG**.
    - **Service connection name**: Type **azure subs**. This name will be referenced in YAML pipelines to access your Azure subscription.
9. Make sure the **Grant access permission to all pipelines** option is unchecked and select **Save**.

Important: The **Grant access permission to all pipelines** option is not recommended for production environments. It is only used in this lab to simplify the configuration of the pipeline.

Note: If you see an error message indicating you don't have the necessary permissions to create a service connection, try again, or configure the service connection manually.

Control Deployments using Release Gates

You will learn how to configure deployment gates and use them to control the execution of Azure Pipelines. You'll configure a release definition with two environments for an Azure Web App, deploying to the DevTest environment only when there are no blocking bugs, and marking the DevTest environment complete only when there are no active alerts in Application Insights.

This lab takes approximately **75** minutes to complete.

Before you start

You need:

- **Microsoft Edge** or an Azure DevOps supported browser
- **Azure subscription:** You need an active Azure subscription or can create a new one
- **Azure DevOps organization:** Create one at Create an organization or project collection if you don't have one
- **Account permissions:** You need a Microsoft account or Microsoft Entra account with:

    - Owner role in the Azure subscription
    - Global Administrator role in the Microsoft Entra tenant
    - For details, see List Azure role assignments using the Azure portal and View and assign administrator roles in Azure Active Directory

About release gates

A release pipeline specifies the end-to-end release process for an application to be deployed across various environments. Deployments to each environment are fully automated using jobs and tasks. It's a best practice to expose updates in a phased manner - expose them to a subset of users, monitor their usage, and expose them to other users based on the experience of the initial set.

Approvals and gates enable you to take control over the start and completion of deployments in a release. You can wait for users to approve or reject deployments manually with approvals. Using release gates, you can specify application health criteria to be met before the release is promoted to the following environment.

Gates can be added to an environment in the release definition from the pre-deployment conditions or the post-deployment conditions panel. Multiple gates can be added to ensure all inputs are successful for the release.

There are 4 types of gates included by default:

- **Invoke Azure Function:** Trigger execution of an Azure Function and ensure successful completion
- **Query Azure Monitor alerts:** Observe configured Azure Monitor alert rules for active alerts
- **Invoke REST API:** Make a call to a REST API and continue if it returns a successful response
- **Query work items:** Ensure the number of matching work items returned from a query is within a threshold

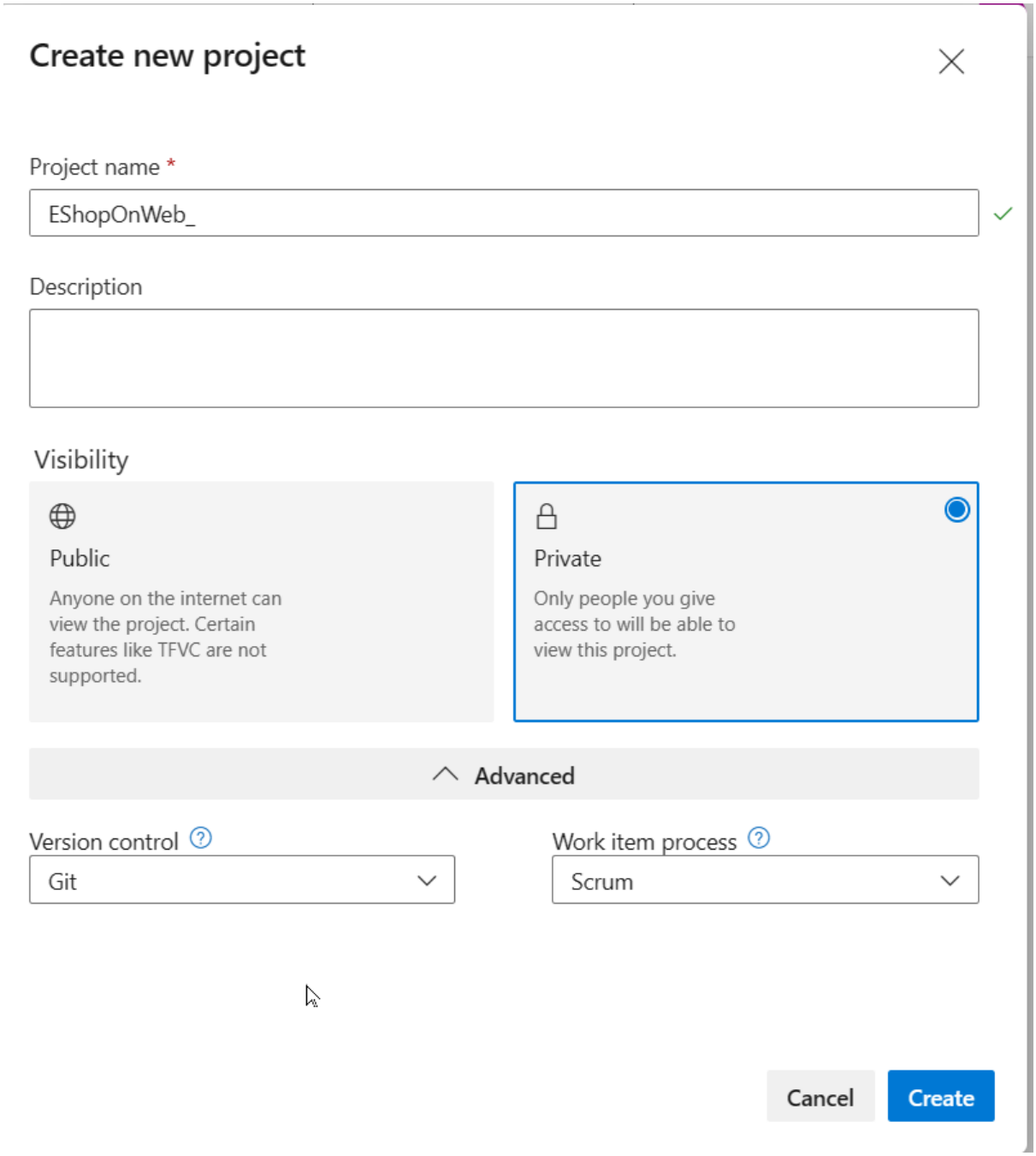
Create and configure the team project

In this **Cloudslice** lab, please skip this task.

First, you'll create an Azure DevOps project for this lab.

1. In your browser, open your Azure DevOps organization
2. Select **New Project**
3. Give your project the name **eShopOnWeb**

4. Leave other fields with defaults
5. Select **Create**



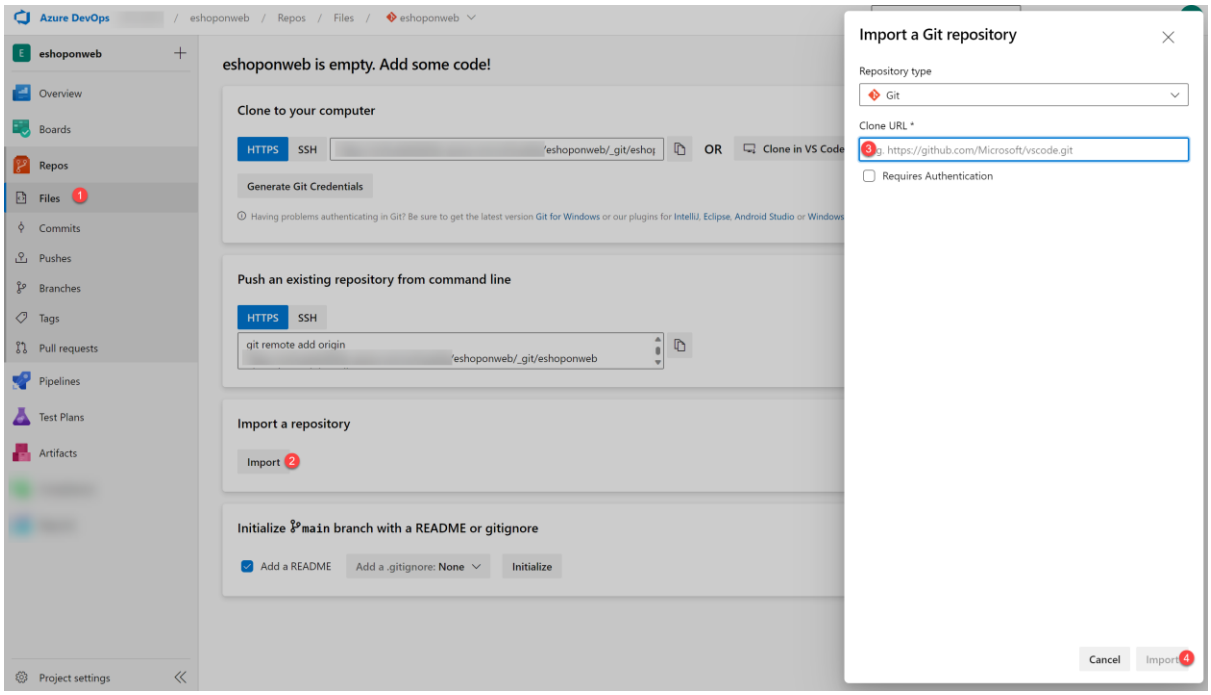
Import the eShopOnWeb Git Repository

In this **Cloudslice** lab, please skip this task.

Next, you'll import the sample repository that contains the application code.

1. In your Azure DevOps organization, open the **eShopOnWeb** project
2. Select **Repos > Files**

3. Select **Import a Repository**
4. Select **Import**
5. In the **Import a Git Repository** window, paste this URL: https://*github.com/MicrosoftLearning/eShopOnWeb.git*
6. Select **Import**



The repository is organized this way:

- **.ado** folder contains Azure DevOps YAML pipelines
- **.devcontainer** folder contains setup to develop using containers
- **infra** folder contains Bicep & ARM infrastructure as code templates
- **.github** folder contains YAML GitHub workflow definitions
- **src** folder contains the .NET 8 website used in the lab scenarios

1. Go to **Repos > Branches**
2. Hover on the **main** branch then select the ellipsis on the right
3. Select **Set as default branch**

Configure CI Pipeline

You'll add a YAML build definition to the project.

1. Navigate to the **Pipelines** section

2. In the **Create your first Pipeline** window, select **Create pipeline**
3. On the **Where is your code?** pane, select **Azure Repos Git (YAML)**
4. On the **Select a repository** pane, select **eShopOnWeb**
5. On the **Configure your pipeline** pane, scroll down and select **Existing Azure Pipelines YAML File**
6. In the **Selecting an existing YAML File** blade, specify:
    - Branch: **main**
    - Path: **.ado/eshoponweb-ci.yml**
7. Select **Continue** to save these settings
8. From the **Review your Pipeline YAML** screen, select **Run** to start the Build Pipeline
9. Wait for the Build Pipeline to complete successfully
10. Go to **Pipelines > Pipelines** and select on the recently created pipeline
11. Select on the ellipsis and **Rename/move** option
12. Name it **eshoponweb-ci** and select **Save**

Create Azure Resources for the Release Pipeline

You'll create two Azure web apps representing the DevTest and Production environments.

Create two Azure web apps

1. From your lab computer, navigate to the **Azure Portal** https*://portal.azure.com*
2. Sign in with the user account that has the Owner role in your Azure subscription
3. In the Azure portal, select the **Cloud Shell** icon (to the right of the search box)
4. If prompted to select either **Bash** or **PowerShell**, select **Bash**

Note: If this is your first time starting Cloud Shell and you see the "You have no storage mounted" message, select your subscription and select "Apply".

5. From the **Bash** prompt, in the **Cloud Shell** pane, run the following command to create a resource group

In order to determine an **Azure** region with capacity, select a region from the dropdown below and then select the **Check Selected Region** button. Once a region is found to have capacity, you may proceed.

eastuseastus2westuswestus2westus3westeuropecanadacentralsoutheastasiaaustraliaeast

Success

Region has capacity: westeurope

REGION='westeurope'

RESOURCEGROUPNAME='az400m03l08-RG'

6. Create an App service plan:

SERVICEPLANNAME='az400m03l08-sp1'

az appservice plan create -g $RESOURCEGROUPNAME -n $SERVICEPLANNAME --sku S1 --location $REGION

7. Create two web apps with unique names:

SUFFIX=61686223

az webapp create -g $RESOURCEGROUPNAME -p $SERVICEPLANNAME -n RGATES$SUFFIX-DevTest

az webapp create -g $RESOURCEGROUPNAME -p $SERVICEPLANNAME -n RGATES$SUFFIX-Prod

Note: Record the name of the DevTest web app. You'll need it later.

Note: If you get an error about the subscription not being registered to use namespace 'Microsoft.Web', run: az provider register --namespace Microsoft.Web and wait for registration to complete.

8. Wait for the provisioning process to complete and close the Cloud Shell pane

Configure an Application Insights resource

1. In the Azure portal, search for **Application Insights** and select it from the results
2. On the **Application Insights** blade, select **+ Create**
3. On the **Basics** tab, specify these settings:

Setting	Value
Resource group	**az400m03l08-RG**

Setting	Value
Name	the name of the DevTest web app from the previous task
Region	the same Azure region where you deployed the web apps

4. Select **Review + create** and then select **Create**
5. Wait for the provisioning process to complete
6. Navigate to the resource group **az400m03l08-RG**
7. In the list of resources, select the **DevTest** web app
8. On the DevTest web app page, in the left menu under **Monitoring**, select **Application Insights**
9. Select **Turn on Application Insights**
10. In the **Change your resource** section, select **Select existing resource**
11. Select the newly created Application Insight resource
12. Select **Apply** and when prompted for confirmation, select **Yes**
13. Wait until the change takes effect

Create monitor alerts

1. From the same **Application Insights** menu, select **View Application Insights Data**
2. On the Application Insights resource blade, under **Monitoring**, select **Alerts**
3. Select **Create > Alert rule**
4. In the **Condition** section, select **See all signals**
5. Type **Requests** and from the results, select **Failed Requests**
6. In the **Condition** section, leave **Threshold** set to **Static** and validate these defaults:
    - Aggregation Type: Count
    - Operator: Greater Than
    - Unit: Count

7. In the **Threshold value** textbox, type **0**
8. Select **Next:Actions**
9. Don't make changes in Actions, and define these parameters under **Details**:

Setting	**Value**
Severity	**2- Warning**
Alert rule name	**RGATESDevTest_FailedRequests**
Advanced Options: Automatically resolve alerts	**cleared**

10. Select **Review+Create**, then **Create**
11. Wait for the alert rule to be created successfully

Note: Metric alert rules might take up to 10 minutes to activate.

Configure the Release Pipeline

You'll configure a release pipeline with deployment gates.

Set Up Release Tasks

1. From the **eShopOnWeb** project in Azure DevOps, select **Pipelines** and then **Releases**
2. Select **New Pipeline**
3. From the **Select a template** window, choose **Azure App Service Deployment** under **Featured** templates
4. Select **Apply**
5. In the Stage window, update the default "Stage 1" name to **DevTest**
6. Close the popup window using the **X** button
7. On the top of the page, rename the pipeline from **New release pipeline** to **eshoponweb-cd**
8. Hover over the DevTest Stage and select the **Clone** button
9. Name the cloned stage **Production**

Note: The pipeline now contains two stages named **DevTest** and **Production**.

10. On the **Pipeline** tab, select the **Add an Artifact** rectangle

11. Select **eshoponweb-ci** in the **Source (build pipeline)** field
12. Select **Add** to confirm the selection
13. From the **Artifacts** rectangle, select the **Continuous deployment trigger** (lightning bolt)
14. Select **Disabled** to toggle the switch and enable it
15. Leave all other settings at default and close the pane

Configure DevTest stage

Due to a known issue, in both **DevTest** and **Production** configuration below, ensure you have selected **Run on agent** and in the **Agent job** pane select **Azure pipelines** in the **Agent pool** field, and **windows-latest** in the **Agent Specification** field.

1. Within the **DevTest Environments** stage, select the **1 job, 1 task** label
2. In the **Azure subscription** dropdown, select your Azure subscription and select **Authorize**
3. Authenticate using the user account with the Owner role in the Azure subscription
4. Confirm the App Type is set to "Web App on Windows"
5. In the **App Service name** dropdown, select the name of the **DevTest** web app
6. Select the **Deploy Azure App Service** task
7. In the **Package or Folder** field, update the default value to: $(System.DefaultWorkingDirectory)/**/Web.zip
8. Open **Application and Configuration Settings** and enter this in **App settings**: -UseOnlyInMemoryDatabase true -ASPNETCORE_ENVIRONMENT Development
9. In the **Run On Agent** Agent settings, navigate to **Agent Selection** and specify the **Agent Pool** as **Hosted / Azure Pipelines** and **windows-latest** for **Agent Specification**
10. Select **Save** and in the Save dialog box, select **OK**

Configure Production stage

1. Navigate to the **Pipeline** tab and within the **Production** Stage, select **1 job, 1 task**
2. Under the Tasks tab, in the **Azure subscription** dropdown, select the Azure subscription you used for the DevTest stage

3. In the **App Service name** dropdown, select the name of the **Prod** web app
4. Select the **Deploy Azure App Service** task
5. In the **Package or Folder** field, update the default value to: $(System.DefaultWorkingDirectory)/**/Web.zip
6. Open **Application and Configuration Settings** and enter this in **App settings**: -UseOnlyInMemoryDatabase true -ASPNETCORE_ENVIRONMENT Development
7. 1. In the **Run On Agent** Agent settings, navigate to **Agent Selection** and specify the **Agent Pool** as **Hosted / Azure Pipelines** and **windows-latest** for **Agent Specification**
8. Select **Save** and in the Save dialog box, select **OK**

Test the release pipeline

1. In the **Pipelines** section, select **Pipelines**
2. Select the **eshoponweb-ci** build pipeline and then select **Run Pipeline**
3. Set **Branch/tag** to **main** and otherwise accept the default settings and select **Run** to trigger the pipeline
4. Wait for the build pipeline to finish

Note: After the build succeeds, the release will be triggered automatically and the application will be deployed to both environments.

5. In the **Pipelines** section, select **Releases**
6. On the **eshoponweb-cd** pane, select the entry representing the most recent release
7. Track the progress of the release and verify that deployment to both web apps completed successfully
8. Switch to the Azure portal, navigate to the **az400m03l08-RG** resource group
9. Select the **DevTest** web app, then select **Browse**
10. Verify that the web page loads successfully in a new browser tab
11. Repeat for the **Production** web app
12. Close the browser tabs displaying the EShopOnWeb web site

Configure Release Gates

You'll set up Quality Gates in the release pipeline.

Configure pre-deployment gates for approvals

1. In the Azure DevOps portal, open the **eShopOnWeb** project
2. In **Pipelines > Releases**, select **eshoponweb-cd** and then **Edit**
3. On the left edge of the **DevTest Environment** stage, select the oval shape representing **Pre-deployment conditions**
4. On the **Pre-deployment conditions** pane, set the **Pre-deployment approvals** slider to **Enabled**
5. In the **Approvers** text box, type and select your Azure DevOps account name

Note: In a real-life scenario, this should be a DevOps Team name alias instead of your own name.

6. **Save** the pre-approval settings and close the popup window
7. Select **Create Release** and confirm by pressing **Create**
8. Notice "Release-2" has been created. Select the "Release-2" link to navigate to its details
9. Notice the **DevTest** Stage is in a **Pending Approval** state
10. Select the **Approve** button to trigger the DevTest Stage